오늘 생성된 파일의 기능

- 날짜: 2026-11-02
- 대상: Next.js 16.4.3, React 19.3.0, `lgcns-fe-next`
- 기준은 현재 소스다. 아래 파일은 오늘 커밋에서 추가된 뒤 아직 저장소에 있는 것이다. 지워진 `component/templates/LoginForm.tsx`와 `component/atoms/Button.tsx`는 없다.

`FormField`가 화면의 입력 하나를 만든다. 그 위 유기체 `LoginForm`이 값을 들고, 페이지의 서버 함수가 제출을 받는다.

1. component/molecules/FormField.tsx

선언은 `'use client'`와 `function FormField`다. 클라이언트 컴포넌트다. `LoginForm`만 이 파일을 import 한다. 브라우저 모듈로 로드되고, `onClick`과 `onChange`는 브라우저에서 실행된다. `<script async>`나 `defer` 태그는 아니다.

인자	하는 일
`id`	입력 id. 오류 문단 id는 `${id}-error`
`name`	폼 전송 때 FormData 키. 지우기 때도 이 이름을 onChange에 넘긴다
`type`	`"text"` 또는 `"password"`. 비밀번호일 때만 보기 버튼을 그린다
`placeholder`	입력 안내 문구. 지우기 버튼 이름에 붙인다
`value`	부모가 가진 현재 문자열. 입력을 통제한다
`error`	없으면 오류 문단은 투명하다. 있으면 aria-invalid와 aria-describedby를 켠다
`isClearable`	참이고 value가 비어 있지 않을 때만 X 버튼을 그린다
`onChange`	글자가 바뀌거나 X를 눌렀을 때 부모 handleChange로 올라간다

`errorId`는 `${id}-error`다. 입력의 `aria-describedby`와 `ErrorMessage`의 `id`가 같은 문자열을 쓴다.

`isVisible`은 `useState(false)`다. 비밀번호를 글자로 보여줄지 여부다. 이 상태는 이 필드 안에만 있고 부모로 올라가지 않는다.

`showClear`는 `Boolean(isClearable && value)`다. 빈 문자열은 거짓이라 X가 없다. `showToggle`은 `type === "password"`다. `inputType`은 비밀번호이면서 `isVisible`이 참이면 `"text"`, 아니면 받은 `type`이다. `type="password"`는 값을 가리고 `type="text"`는 보여 준다.

`handleClear`는 브라우저 ChangeEvent를 만들지 않는다. `{ target: { name, value: "" } }`만 만들어 `onChange`에 넘긴다. 부모는 `event.target.name`과 `event.target.value`만 읽는다.

1. 바깥 div.
2. `className="relative"`인 div. 버튼을 입력 위에 겹치기 위한 기준이다.
3. Input. value, name, required, onChange를 넘긴다. className은 버튼이 둘이면 `"pr-14"`, 하나면 `"pr-8"`, 없으면 undefined다.
4. showToggle일 때 type="button" 버튼. aria-pressed는 isVisible이다. 보이면 EyeOff와 "비밀번호 숨기기", 가려져 있으면 Eye와 "비밀번호 보기"다. 클릭은 setIsVisible로 참과 거짓을 뒤집는다.

5. showClear일 때 type="button" 버튼. X 아이콘이다. 비밀번호 보기 버튼이 있으면 right-7, 없으면 right-1.5다.
6. ErrorMessage.

폼 안의 button은 type을 빼면 제출 버튼이 된다. 두 버튼은 type="button"이라 폼을 보내지 않는다. 이 파일은 FormData를 만들지 않고 서버 함수를 부르지 않는다.

2. component/atoms/ErrorMessage.tsx

선언은 `function ErrorMessage`다. 파일에 `"use client"`는 없다. FormField가 import 하므로 클라이언트 그래프에 실려 브라우저에서 함께 실행된다.

인자 id는 문단 id다. error는 없을 수 있는 문자열이다. `if (!error) return null`은 주석이다. 그래서 오류가 없어도 함수는 항상 p를 반환한다. role="alert"다. className은 "text-red-500 py-4"다. style.opacity는 error가 있으면 1, 없으면 0이다. style.transition은 "opacity 0.3s ease-in-out"이다. 글자가 없어도 py-4 높이는 남는다. 자식은 error 한 개다.

3. component/organisms/LoginForm.tsx

선언은 `"use client"`와 `function LoginForm`이다. LoginSection이 import 한다. 인자 handleSubmit은 `(formData: FormData) => void`다. 페이지의 서버 함수가 이 인자로 들어온다. 이 파일 안에서 "use server"를 적지 않는다.

LoginFieldName은 UserPayloadType의 키라 "userId" 또는 "password"다. payload의 처음 값은 loginInitial.ts의 initialValues다. errors의 처음 값은 loginError.ts의 emptyErrors다. isValid는 loginSchema.safeParse(payload).success다. 렌더마다 다시 계산하고, 따로 상태로 저장하지 않는다.

handleChange는 event.target에서 name과 value를 읽는다. name이 두 필드가 아니면 반환한다. 맞으면 브라우저 콘솔에 name과 value를 찍고, payload의 그 키만 바꾸고, loginFieldMessage(field, value) 결과로 errors의 그 키만 바꾼다. 다른 필드의 오류는 그대로 둔다.

반환은 next/form의 Form이다. action={handleSubmit}이다. 이 컴포넌트는 action이 함수이면 그 서버 함수를 실행한다. 자식은 아이디 FormField(type="text", isClearable={true}), 비밀번호 FormField(type="password", isClearable={true}), 제출 Button이다. 제출 버튼은 type="submit"이고 disabled={!isValid}다.

4. component/templates/LoginSection.tsx

선언은 `function LoginSection`이다. "use client"가 없다. 서버 컴포넌트다. app/(auth)/login/page.tsx가 import 한다. 함수 본문은 브라우저 번들로 내려가지 않는다. 인자 handleSubmit을 그대로 LoginForm에 넘긴다. 반환은 section 하나다. 상태, Zod, 입력은 이 파일에 없다.

5. schema/loginSchema.ts

"use client"가 없다. LoginForm이 import 하므로 클라이언트 그래프에서 실행된다. 서버 함수 handleSubmit은 이 파일을 부르지 않는다.

loginSchema는 z.object다. userId와 password는 각각 z.string().min(1, 메시지)다. 길이가 1보다 작으면 그 문자열이 이슈 메시지가 된다. LoginFieldName은 keyof UserPayloadType이다.

loginFieldMessage(field, value)는 loginSchema.shape[field].safeParse(value)를 한다. 성공하면 ""를 반환한다. 실패하면 result.error.issues[0]?.message를 반환하고, 그것도 없으면 ""다. 객체 전체가 아니라 그 필드 스키마만 검사하므로, 한쪽만 비어도 다른 쪽 메시지가 이 함수 결과로 섞이지 않는다. loginFieldMessage는 이름 있는 내보내기다. loginSchema는 기본 내보내기다.

6. types/request/RequestType.ts

interface UserPayloadType은 userId: string, password: string이다. export type { UserPayloadType }로 이름 있게 내보낸다. 런타임 값을 만들거나 검사하지 않는다. loginSchema, loginInitial, loginError, LoginForm이 이 모양을 쓴다.

7. types/initial/loginInitial.ts

initialValues는 UserPayloadType이고 두 필드가 ""다. export default initialValues다. LoginForm의 useState(initialValues) 처음 값이다. 함수는 없다.

8. types/error/loginError.ts

LoginFieldName은 keyof UserPayloadType이다. emptyErrors는 Record<LoginFieldName, string>이고 두 값이 ""다. export const emptyErrors로 이름 있게 내보낸다. 기본 내보내기는 없다. LoginForm의 오류 상태 처음 값이다. 함수는 없다.

9. app/(auth)/client-login/page.tsx

선언은 `'use client'`와 `function ClientLogin`이다. 라우트 그룹 (auth)는 URL에 없다. 주소는 /client-login이다. 페이지 전체가 클라이언트 컴포넌트라 브라우저 모듈로 로드된다.

userId, password는 각각 useState("")다. handleUserIdChange는 e.target.value를 브라우저 콘솔에 찍고 setUserId를 호출한다. handlePasswordChange는 같은 일을 비밀번호 상태에 한다. handleSubmit은 e.preventDefault()로 폼의 기본 전송을 막는다. 그다음 상태 두 개를 브라우저 콘솔에 찍는다. FormData를 만들지 않고 서버 함수를 부르지 않는다.

반환은 HTML form onSubmit={handleSubmit}이다. 입력 둘은 value와 onChange만 있고 name은 없다. name이 없으므로, 설령 기본 제출이 일어나도 폼 데이터 셋에 들어가지 않는다. 제출 버튼에 disabled 조건은 없다. Zod, X 버튼, 보기 버튼은 없다.

10. components/ui/input.tsx

선언은 `function Input`이다. "use client"는 이 파일에 없다. @base-ui/react/input의 Input.js 첫 줄이 'use client'라 이 입력을 그리는 쪽은 클라이언트 경계가 된다. export { Input }이다. 기본 내보내기는 없다.

className과 type을 빼고 나머지 props를 InputPrimitive에 넘긴다. data-slot="input"을 붙인다. className은 cn으로 기본 클래스와 호출자가 준 className을 합친다. FormField가 넘기는 id, name, value, required, aria-*, onChange는 ...props로 들어간다. 이 파일은 값을 저장하지 않는다.

11. component/atoms/CustomButton.tsx

선언은 `function CustomButton`이다. "use client"는 이 파일에 없다. @base-ui/react/button이 클라이언트 경계다. export { CustomButton, buttonVariants }다. 기본 내보내기는 없다.

buttonVariants는 cva로 만든 클래스 조합이다. variant에는 default, lgcns, outline, secondary, ghost, destructive, link가 있다. size에는 default, xs, sm, lg, icon, icon-xs, icon-sm, icon-lg가 있다. 기본 variant와 size는 둘 다 "default"다.

CustomButton은 className, variant = "default", size = "default", 나머지 props를 받는다. ButtonPrimitive에 data-slot="button"과 cn(buttonVariants({ variant, size, className }))을 넘긴다. 로그인 폼은 이 파일을 쓰지 않는다. 로그인 제출 버튼은 components/ui/button.tsx의 Button이다.

12. 호출 순서

/login에서 글자를 입력하면 FormField의 Input이 onChange를 부르고, LoginForm.handleChange가 payload와 errors를 바꾼다. loginFieldMessage는 그 필드만 검사한다. X는 같은 handleChange에 빈 문자열을 넘긴다. 보기 버튼은 FormField의 isVisible만 바꾼다. 제출은 name이 있는 입력으로 FormData를 만들고, 페이지의 handleSubmit이 서버에서 formData.get을 한다. /client-login은 이 연쇄에 들어가지 않는다.

- https://react.dev/reference/react/useState
- https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input
- https://developer.mozilla.org/en-US/docs/Web/HTML/Element/button
- https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Reference/Attributes/aria-pressed
- https://developer.mozilla.org/en-US/docs/Web/API/Event/preventDefault
- https://developer.mozilla.org/en-US/docs/Web/API/FormData
- https://nextjs.org/docs/app/api-reference/components/form
- https://zod.dev